

Fast Core Tool (FCT) - Offline Documentation

Table of Contents

- 1. **Introduction**
- 2. **Setup Guide**
 - 2.1. Initializing FCT
 - 2.2. Quick Setup Tutorial
 - 2.3. Manual Setup via Hub
- 3. **Configuring Systems**
 - 3.1. Setting Up Inputs
 - 3.2. Configuring Localization
- 4. **Script Reference**
 - 4.1. FCT.Device.GameInput
 - 4.2. FCT.Gameplay.SimplePool
 - 4.3. FCT.FSM.StateMachine
 - 4.4. FCT.Localization.LocalizationManager

1. Introduction

Fast Core Tool (FCT) is a professional, modular framework for Unity designed to jumpstart game development. It handles boilerplate systems seamlessly, including Inputs, State Machines, Localization, and Object Pooling. This documentation provides a comprehensive guide to setting up and integrating FCT into your Unity projects.

2. Setup Guide

2.1. Initializing FCT

After importing the FCT package into your Unity project, you can access all central tools through the Unity menu.

1. Navigate to the top Unity menu.
2. Click ****FCT -> Core Wizard****. This opens the central command center for FCT.

2.2. Quick Setup Tutorial

The quickest way to get started is by using the guided tutorial provided within the tool.

1. Open the **Core Wizard** (FCT -> Core Wizard).
2. On the **Home** tab, click the blue **"Start Quick Tutorial"** button.
3. Follow the on-screen guided mode. It will walk you through setting up all core systems, including Input, Object Pool, State Machine, Camera, and Localization directly into your current active scene.

2.3. Manual Setup via Hub

If you prefer to configure systems individually or integrate them into an existing scene architecture:

1. Open the **Core Wizard** and navigate to the **Setup Hub** tab.
2. Click **Setup SimplePool** to instantiate the object pooling manager into your scene.
3. Click **Generate GameManager** to create a boilerplate State Machine script ready for custom game logic.
4. Click **Setup Localization** to initialize the `LocalizationManager` and its required database file.

3. Configuring Systems

3.1. Setting Up Inputs

FCT leverages the Unity New Input System. To configure your actions:

1. Double-click your `.inputactions` file in the project and define your input actions (e.g., "Jump", "Fire").
2. Create a new Input Config by right-clicking in the Project window and selecting **Create -> FCT -> Input Config**.
3. Assign your `.inputactions` file to the newly created Input Config.
4. Click the **"Sync Actions"** button on the Input Config inspector to link the actions to FCT.

3.2. Configuring Localization

FCT allows you to synchronize localization directly from a Google Sheets CSV export.

1. In the **Core Wizard**, navigate to the **Config** tab.
2. Paste your Google Sheets CSV Export URL into the provided field.
3. Click **"Sync Now"**. All translations will be downloaded and saved into the project database.

4. To localize UI elements, add the ``LocalizeText`` component to any `TextMeshPro` element and select your translation key from the dropdown menu.

4. Script Reference

The following section outlines the primary classes available in FCT for developers who require custom coding.

4.1. `FCT.Device.GameInput`

The centralized input hub for the framework. Provides dynamic access to all active input values.

- * `**`GameInput.Instance`**`: Singleton access to the manager.
- * `**`bool GetButtonDown(string actionName)`**`: Returns true during the frame the user pressed down the virtual button.
- * `**`bool GetButton(string actionName)`**`: Returns true while the virtual button is held down.
- * `**`bool GetButtonUp(string actionName)`**`: Returns true during the frame the user released the virtual button.
- * `**`float GetFloat(string actionName)`**`: Returns the float value of an axis.
- * `**`Vector2 GetVector2(string actionName)`**`: Returns the `Vector2` value (e.g., for movement or joystick axes). Also automatically injects a virtual joystick value if the action name is "Move" and the joystick is active.

4.2. `FCT.Gameplay.SimplePool`

A highly optimized Object Pool system supporting pre-warming and safe despawning.

- * `**`SimplePool.Instance`**`: Singleton access.
- * `**`void Prewarm(GameObject prefab, int count, Transform parent = null)`**`: Pre-instantiates a number of objects to avoid FPS drops during gameplay.
- * `**`GameObject Spawn(GameObject prefab, Vector3 position, Quaternion rotation, Transform parent = null)`**`: Retrieves an object from the pool or instantiates a new one if the pool is empty. Automatically calls ``OnSpawn()`` on any attached ``IPoolable`` interfaces.
- * `**`void Despawn(GameObject obj)`**`: Returns an object to the pool, disabling it and calling ``OnDespawn()`` on attached ``IPoolable`` interfaces.
- * `**`void Clear()`**`: Destroys all currently pooled objects and clears the pool dictionary to free memory.

4.3. FCT.FSM.StateMachine<T>

A generic, scalable, and safe state machine.

* **`StateMachine(T owner)`**: Constructor that assigns the owner of the state machine.

* **`IState<T> CurrentState`**: Property to get the currently active state.

* **`void ChangeState(IState<T> newState)`**: Transitions to a new state. Calls `OnExit` on the previous state and `OnEnter` on the new state.

* **`void Update()`, `void FixedUpdate()`, `void LateUpdate()`**: Tick methods that should be called from Unity's respective update loops to process the current state's logic.

4.4. FCT.Localization.LocalizationManager

Manages languages and dynamic string lookups.

* **`LocalizationManager.Instance`**: Singleton access.

* **`event Action OnLanguageChanged`**: Event triggered whenever the active language changes.

* **`void SetLanguage(Language lang)`**: Changes the active language and saves the preference to `PlayerPrefs`.

* **`string GetLocalText(string key)`**: Retrieves the localized string for a specific key based on the current language and platform.